Digital Signal Processing

Digital FIR Filter Design Project

Authors:

Salma Hatem
Omar Ayman
Nada Sobhy
Aser Osama

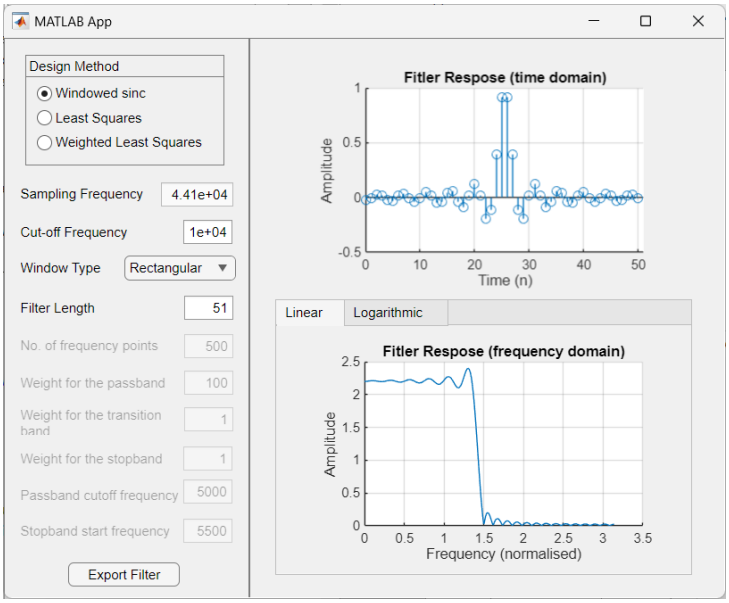
Date:

December 2024

Contents

1 Part 1 - Filter Design Tool

1.1 Explaining the GUI modifiable parameters



1.1.1 Filter Length

Regardless of which designing method is used, changing the filter length (or number of coefficients) greatly affects the filter response. Generally, increasing the length will result in sharper, usually better, response. The length of the filter is restricted to be an odd number, and our tool will automatically add 1 to the filter length in case the user inputs an even number. Using large filter lengths increases the complexity of the filter significantly, thus this is a trade off to be considered while designing any filter.

1.1.2 Window Type (Windowed Sync Only)

Using different window types affects many of the filter parameters such as transition time, stopband attenuation and passband ripple. For rectangular windows, the transition time is very small, but it results in smaller stopband attenuation and high passband ripple compared to other windows. Kaiser window is similar to rectangular window, though it has better stopband attenuation and passband ripple. As for Blackman and Chebyshev windows, they have significantly better stopband attenuation and passband ripple and an overall smoother response. It is noted from the filter response in logarithmic scale that their side lobes have a much smaller amplitude than rectangular and Kaiser windows (at around 10^{-4} and 10^{-5} compared to 10^{-1}). However, they both have a larger transition time with Blackman being slightly better.

1.1.3 Frequency Points

While the number of frequency points does not really affect the filter parameters mentioned before, which are here mainly controlled by the length of the filter. Instead, it helps the user control the frequency resolution in the representation of the graph.

1.1.4 Weights & Specific frequencies (Weighted Least Squares Only)

Here the user has more control over the filter parameters through adjusting the weights and stopband start frequency. Each weight mainly controls part of the filter response while slightly affecting other parameters. For example, giving higher value for the weight for the passband will improve the passband ripples but it'll worsen the transition time and stopband attenuation. Similarly, weights for stopband improves stopband attenuation at the cost of passband ripples and transition time. Lastly, weight for transition band, as well as the start frequency of the stopband, improves the transition time while resulting in higher overshoot and worse passband ripples and stopband attenuation.

1.2 Windowed Sinc

The first low pass filter implementation we added was windowed sinc with multiple choices for the windows.
The tool can be used to generate a windowed sinc filter using rectangular, Blackman, Chebyshev, and Kaiser windows through a simple drop down menu and the filter length, sampling frequency, and cut-off frequencies can all be entered using input boxes.

1.2.1 Windowed Sinc GUI

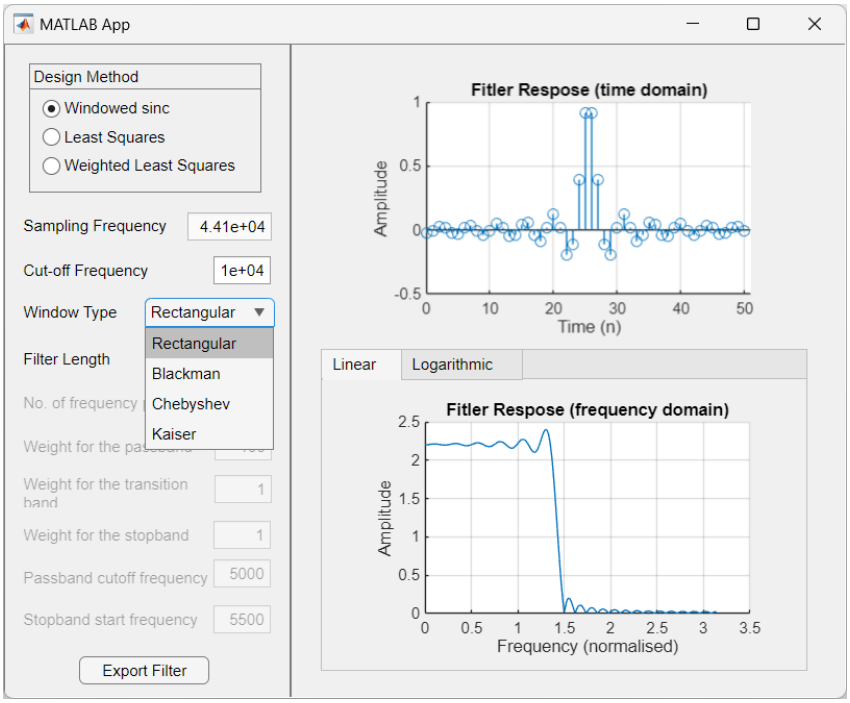


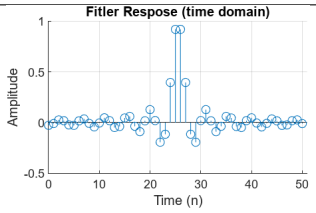
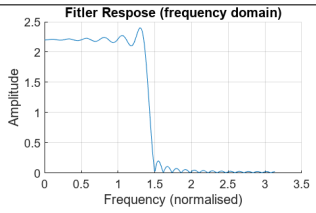
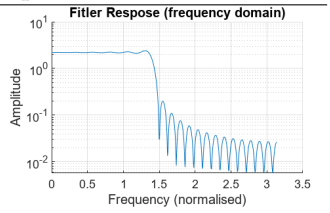
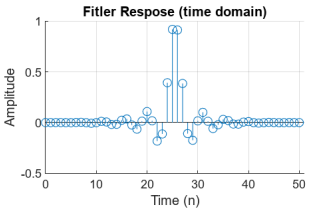
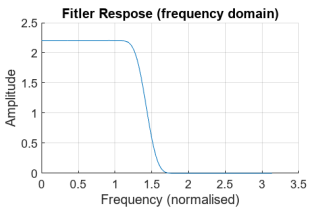
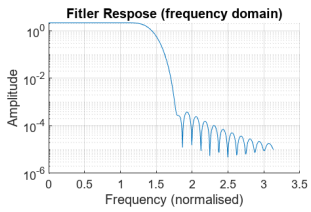
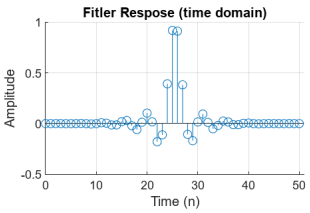
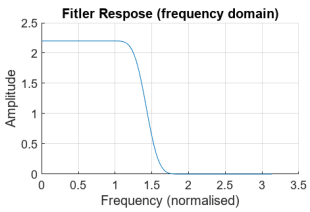
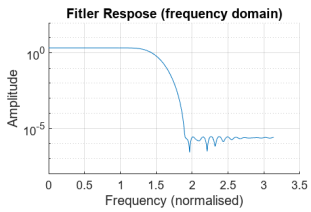
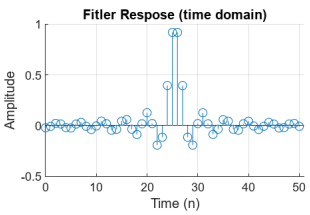
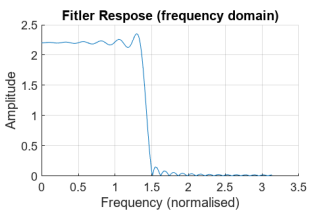
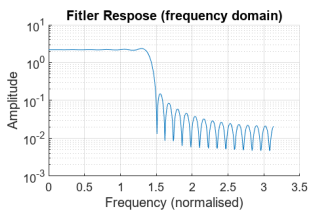
Figure 1: GUI for Windowed Sinc

1.2.2 Windowed Sinc Code

```
function [n,coeff,f,H] = windowed_sinc(L,window_type,fc)

fs = 44100; % Sampling rate in Hz
n = 0:L-1; % L has to be odd

% sinc(x) = sin(pi*x)/pi*x
sinc_signal = sinc(2*fc/fs*(n-L/2));

% creating window
switch(window_type)
    case 'Rectangular'
        window = rectwin(L);
    case 'Blackman'
        window = blackman(L);
    case 'Chebyshev'
        window = chebwin(L);
    case 'Kaiser'
        window = kaiser(L,1);
end
% multiplying sinc by window
coeff = sinc_signal .* window';

% getting response in frequency domain
[H, f] = freqz(coeff, 1);
end
```

1.2.3 Different Windows' Plots

Table 1: Low Pass Filter Analysis with Different Window Functions

Window Type	Impulse Response	Frequency Response	Log Frequency Response
Rectangular			
Blackman			
Chebyshev			
Kaiser			

1.3 Least Squares

The second low pass filter implementation we added was the least squares filter.
Here you have the same choices as the Windowed sinc minus the window type which is replaced by a new option that is "Number of frequency points".

1.3.1 Least Squares GUI

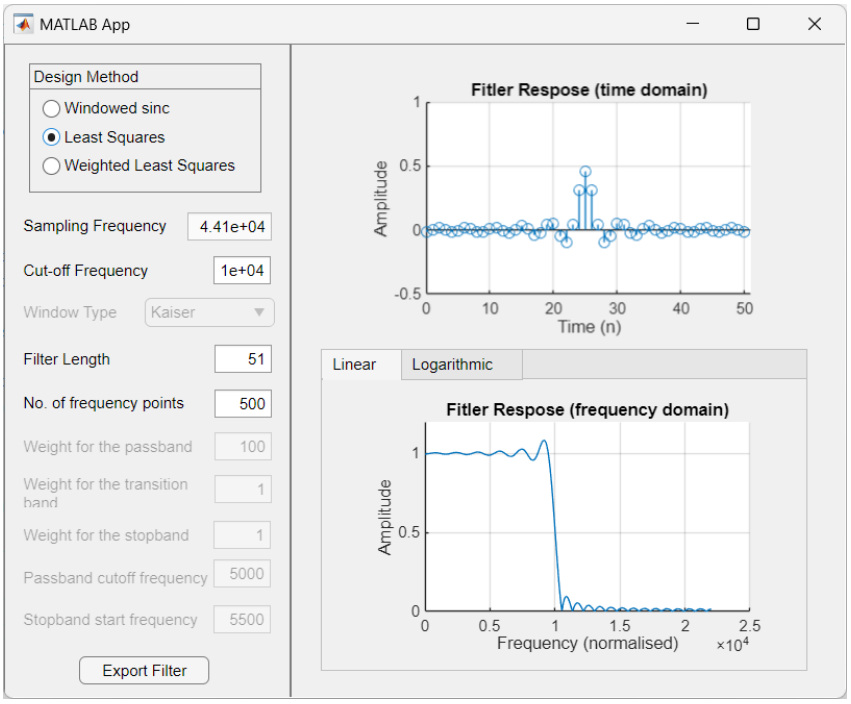


Figure 2: GUI for Least Squares

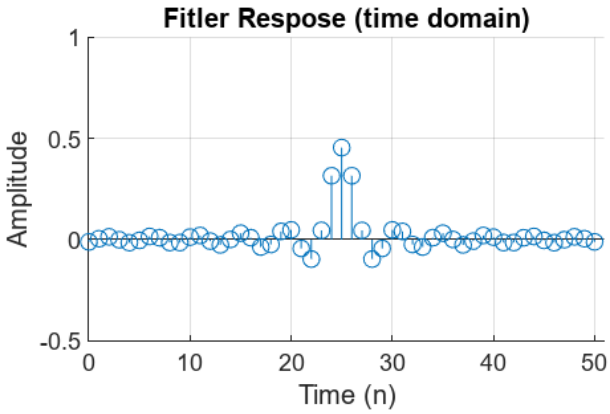
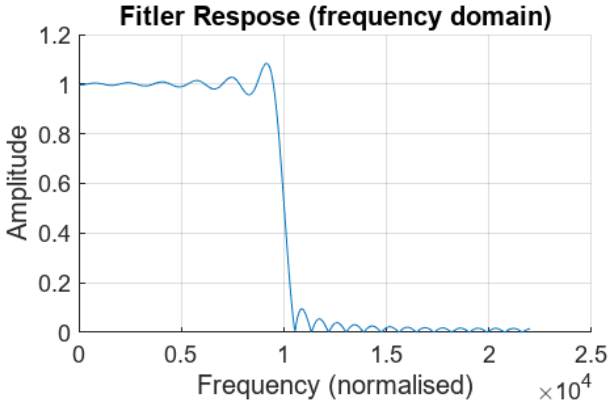
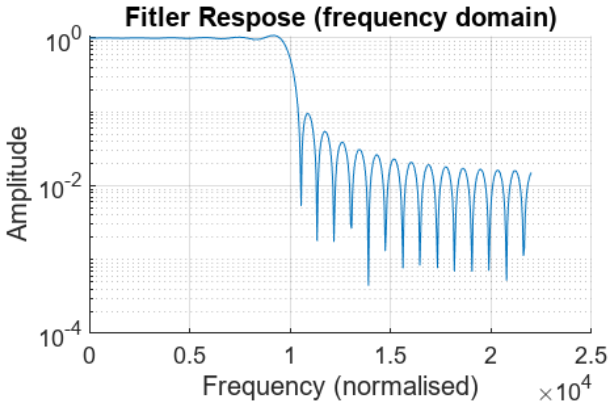
1.3.2 Least Squares Code

```
function [n, coeff, f, H] = ls_fir_filter(N, k, f_cutoff)
%   Inputs:
%       N           - Desired filter length (must be odd and positive)
%       k           - Number of frequency points (must be > (N-1)/2 + 1)
%       f_cutoff    - Desired cutoff frequency in Hz (must be < fs/2)
%
%   Outputs:
%       n           - Filter index vector (0 to N-1)
%       coeff       - Filter coefficients (1 x N vector)
%       f           - Frequency vector in Hz (1 x k vector)
%       H           - Frequency response (complex vector, 1 x k)
%
%   Example:
%       [n, coeff, f, H] = ls_fir_filter(11, 50, 2000);
    %% 1. Initialize and Validate Inputs

    % Define Sampling Rate (Fixed at 44100 Hz)
    fs = 44100;                 % Sampling rate in Hz

    % Default Parameters
    default_N = 15;             % Default filter length (odd and
       positive)
    default_f_cutoff = 5000;    % Default cutoff frequency in Hz

    % Validate and Assign Filter Length N
    if N <= 0
        warning('Invalid filter length N. Using default N = %d.',
           default_N);
        N = default_N;
    end
    if mod(N, 2) == 0
        warning('Filter length N is even. Incrementing N by 1 to make
           it odd.');
        N = N + 1;
    end

    L = (N-1)/2;                % Half-length for symmetric filter
       design
    n = 0:1:N-1;                % Filter index vector

    % Validate and Assign Number of Frequency Points k
    if k <= 0 || k <= (L + 1)
        warning('Invalid or insufficient number of frequency points k.
           Using default k = %d.', (N - 1)/2 + 2);
        k = (N - 1)/2 + 2;
    end

    % Validate and Assign Cutoff Frequency f_cutoff
    if f_cutoff <= 0 || f_cutoff >= fs/2
        warning('Invalid or missing cutoff frequency f_cutoff. Using
           default f_cutoff = %d Hz.', default_f_cutoff);
        f_cutoff = default_f_cutoff;
    end

    %% 2. Define Frequency Design Grid

```

```
    w = pi * linspace(0, 1, k); % Frequency vector from 0 to pi (rad/
        sample)

    %% 3. Desired Frequency Response (Low-Pass)

    Hd = zeros(1, k);                  % Initialize desired frequency
        response
    cut_off = 2 * pi * f_cutoff / fs;  % Normalize cutoff frequency (
        rad/sample)
    Hd(w <= cut_off) = 1;               % Passband: 1 up to cutoff
        frequency, 0 otherwise

    %% 4. Construct the F Matrix for Least Squares

    % Construct cosine terms for the F matrix
    cos_terms = cos(w' * (1:L));

    % Assemble F matrix
    F = [ones(k,1), 2*cos_terms];

    %% 5. Solve the Least Squares Problem

    % Solve F * h_half' = Hd' for h_half
    h_half = F \ Hd';             % Least squares solution (column
        vector)
    h_half = h_half';             % Convert to row vector

    % h_half(1) = h(0), h_half(2) = h(1), ..., h_half(L+1) = h(L)

    %% 6. Construct Full Symmetric Filter Coefficients

    % Mirror the coefficients to ensure symmetry: h(k) = h(-k)
    h_full = [fliplr(h_half(2:end)), h_half]; % Length: N
    coeff = h_full;

    %% 7. Compute Frequency Response
    [H, f] = freqz(coeff, 1, k, fs);     % Frequency response and
        frequency vector in Hz

end
```

1.3.3 Least Squares Plots

Table 2: Low Pass Filter Analysis with Least Squares Method

Plot Type	Image
Impulse Response	
Frequency Response	
Log Frequency Response	

1.4 Wieghted Least Squares

The third and final low pass filter implementation we added was the weight least squares filter.
Here you have the same inputs as the normal least squares filter as expected as well as the ability to enter your “Pass band”, “Transition band”, and “Stop band” weights as well as the “Pass band cutoff frequency” and the “Stop Band Stop Frequency” to control the sharpness of your filters transition at the cost of overshoot.

1.4.1 Weighted Least Squares GUI

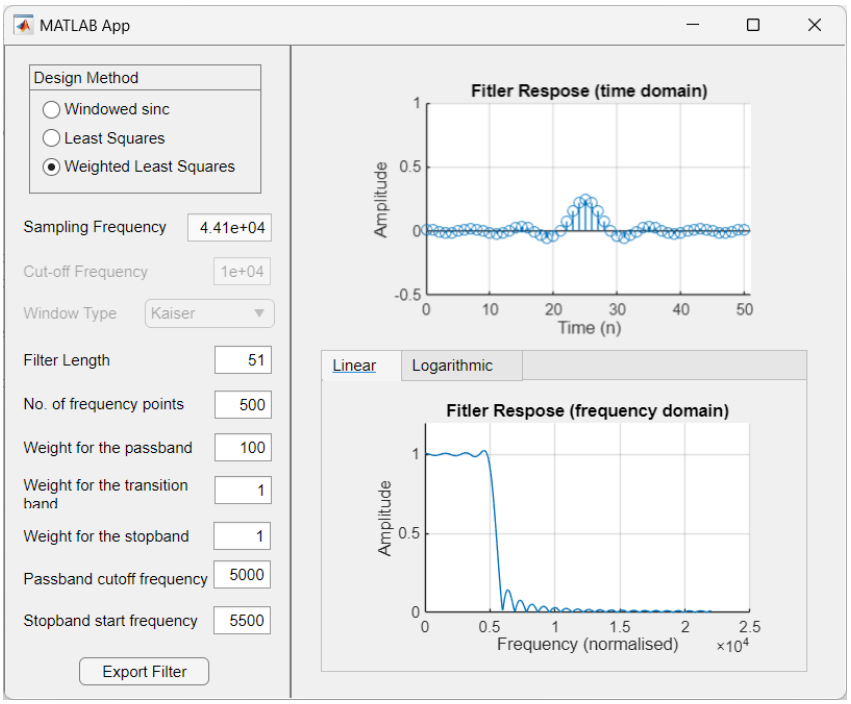


Figure 3: GUI for Weighted Least Squares

1.4.2 Weighted Least Squares Code

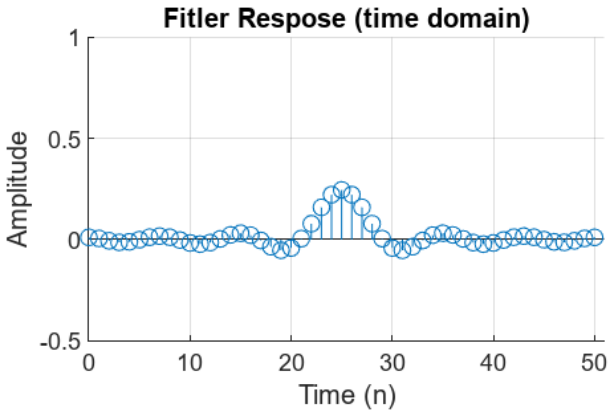
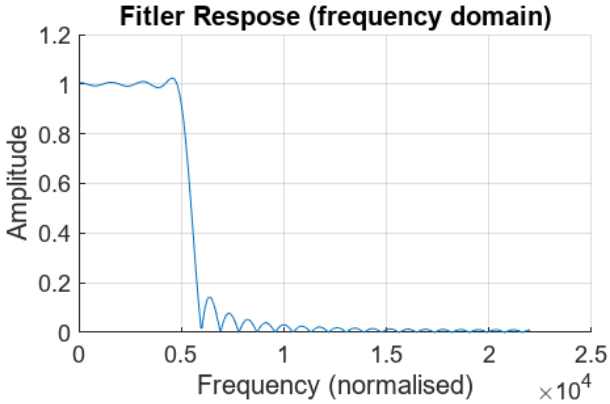
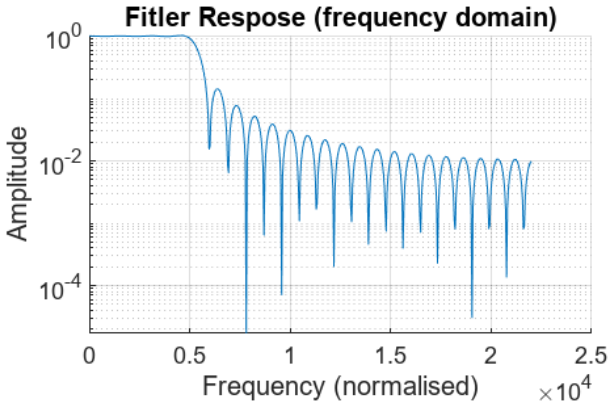
```
function [n, coeff, f, H] = wls_fir_filter(W_p, W_t, W_s, N, k, f_pass, f_stop)
%   Inputs:
%       W_p         - Weight for the passband (positive scalar)
%       W_t         - Weight for the transition band (positive scalar)
%       W_s         - Weight for the stopband (positive scalar)
%       N           - Desired filter length (must be odd and positive)
%       k           - Number of frequency points (must be > (N-1)/2 + 1)
%       f_pass      - Passband cutoff frequency in Hz (must be < fs/2)
%       f_stop      - Stopband start frequency in Hz (must be > f_pass and < fs/2)
%
%   Outputs:
%       n           - Filter index vector (0 to N-1)
%       coeff       - Filter coefficients (1 x N vector)
%       f           - Frequency vector in Hz (1 x k vector)
%       H           - Frequency response (complex vector, 1 x k)
%
%   Example:
%       [n, coeff, f, H] = ls_fir_filter(100, 1, 1, 301, 500, 5000, 5500);

    %% 1. Initialize and Validate Inputs

    % Define Sampling Rate (Fixed at 44100 Hz)
    fs = 44100;                 % Sampling rate in Hz

    % Default Parameters (in case inputs are missing or invalid)
    default_W_p = 1;            % Default weight for passband
    default_W_t = 1;            % Default weight for transition band
    default_W_s = 1;            % Default weight for stopband
    default_N = 301;            % Default filter length (odd and positive)
    default_f_pass = 5000;      % Default passband cutoff frequency in Hz

    % Validate and Assign Weight for Passband W_p
    if  W_p <= 0
        warning('Invalid or missing passband weight W_p. Using default W_p = %d.', default_W_p);
        W_p = default_W_p;
    end

    % Validate and Assign Weight for Transition Band W_t
    if W_t <= 0
        warning('Invalid or missing transition band weight W_t. Using default W_t = %d.', default_W_t);
        W_t = default_W_t;
    end

    % Validate and Assign Weight for Stopband W_s
    if W_s <= 0
        warning('Invalid or missing stopband weight W_s. Using default W_s = %d.', default_W_s);
        W_s = default_W_s;
    end
```

```
49
50      % Validate and Assign Filter Length N
51      if N <= 0
52          warning('Invalid or missing filter length N. Using default N =
                %d.', default_N);
53          N = default_N;
54      end
55      if mod(N, 2) == 0
56          warning('Filter length N is even. Incrementing N by 1 to make
                it odd.');
57          N = N + 1;
58      end
59
60      L = (N-1)/2;                % Half-length for symmetric filter
            design
61      n = 0:1:N - 1;                % Filter index vector
62
63      % Validate and Assign Number of Frequency Points k
64      if k <= 0 || k <= L + 1
65          warning('Invalid or insufficient number of frequency points k.
                Using default k = %d.', (N-1)/2 + 1);
66          k = (N - 1)/2 + 2;
67      end
68
69      % Validate and Assign Passband Cutoff Frequency f_pass
70      if f_pass <= 0 || f_pass >= fs/2
71          warning('Invalid or missing passband cutoff frequency f_pass.
                Using default f_pass = %d Hz.', default_f_pass);
72          f_pass = default_f_pass;
73      end
74
75      % Validate and Assign Stopband Start Frequency f_stop
76      if f_stop <= f_pass || f_stop >= fs/2
77          warning('Invalid or missing stopband start frequency f_stop.
                Using default f_stop = %d Hz.', f_pass + 2000);
78          f_stop = f_pass + 2000;
79      end
80
81      %% 2. Define Frequency Design Grid
82
83      w = pi * linspace(0, 1, k); % Frequency vector from 0 to pi (rad/
            sample)
84
85      %% 3. Desired Frequency Response (Low-Pass)
86
87      Hd = zeros(1, k);                  % Initialize desired frequency
            response
88      cut_off_pass = 2 * pi * f_pass / fs;   % Normalize passband cutoff
            frequency (rad/sample)
89      cut_off_stop = 2 * pi * f_stop / fs;   % Normalize stopband start
            frequency (rad/sample)
90      Hd(w <= cut_off_pass) = 1;                % Passband: 1 up to
            passband cutoff, 0 otherwise
91
92      %% 4. Define Weights for Each Band
93
94      % Initialize weight vector
95      W = zeros(1, k);
```

```
96
97      % Assign weights based on frequency bands
98      W(w <= cut_off_pass) = W_p;                          % Passband
99      W(w > cut_off_pass & w < cut_off_stop) = W_t;    % Transition band
100     W(w >= cut_off_stop) = W_s;                          % Stopband
101
102     % Construct diagonal weight matrix
103     W_matrix = diag(W);                                   % Diagonal matrix
            with weights
104
105     %% 5. Construct the F Matrix for Weighted Least Squares
106
107     % Construct cosine terms for the F matrix
108     cos_terms = cos(w' * (1:L));
109
110     % Assemble F matrix
111     F = [ones(k,1), 2*cos_terms];
112
113     %% 6. Solve the Weighted Least Squares Equation Precisely
114
115     % Compute F' * W * F
116     FWF = F' * W_matrix * F;
117
118     % Compute F' * W * H
119     FWH = F' * W_matrix * Hd';
120
121     % Solve for h_half: (F'WF) h_half = F'WH
122     h_half = FWF \ FWH;
123     h_half = h_half';           % Convert to row vector
124
125     % h_half(1) = h(0), h_half(2) = h(1), ..., h_half(L+1) = h(L)
126
127     %% 7. Construct Full Symmetric Filter Coefficients
128
129     % Mirror the coefficients to ensure symmetry: h(k) = h(-k)
130     h_full = [fliplr(h_half(2:end)), h_half]; % Length: N
131     coeff = h_full;
132
133     %% 8. Compute Frequency Response
134
135     [H, f] = freqz(coeff, 1, k, fs);     % Frequency response and
            frequency vector in Hz
136
137 end
```

1.4.3 Weighted Least Squares Plots

Table 3: Low Pass Filter Analysis with Least Squares Method

Plot Type	Image
Impulse Response	
Frequency Response	
Log Frequency Response	

2 Part 2 - Audio Filtering

In this section we will test out our filter design by loading in a normal audio into Matlab, adding high frequency noise, and attempting to remove that noise using a filter exported out using our app. We will confirm the success of our filtering by viewing the frequency spectrum of the signal as well as listening to the audio at multiple stages of our process.

2.1 Loading Audio & preprocessing the audio

2.1.1 Loading in the audio and extracting the sampling frequency

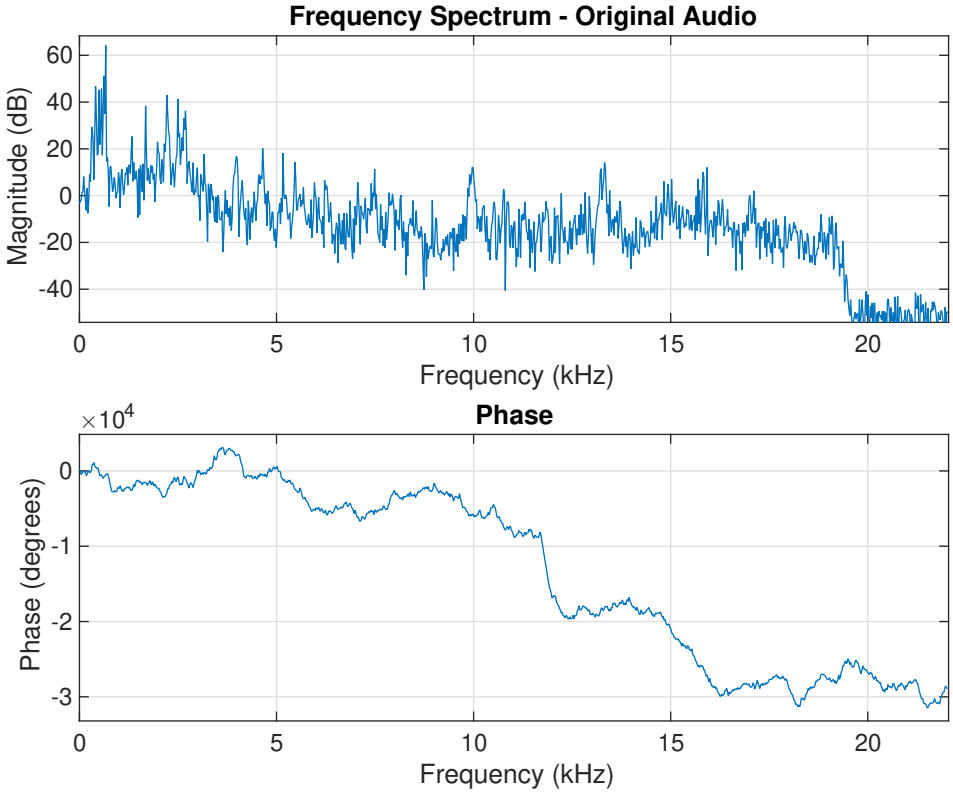
```
%Import the audio and convert to mono-channel if required
[audio, Fs] = audioread('.\bells-melody-loop-266598.wav');
if size(audio, 2) == 2
    audio = mean(audio, 2);
end

disp(Fs);
```

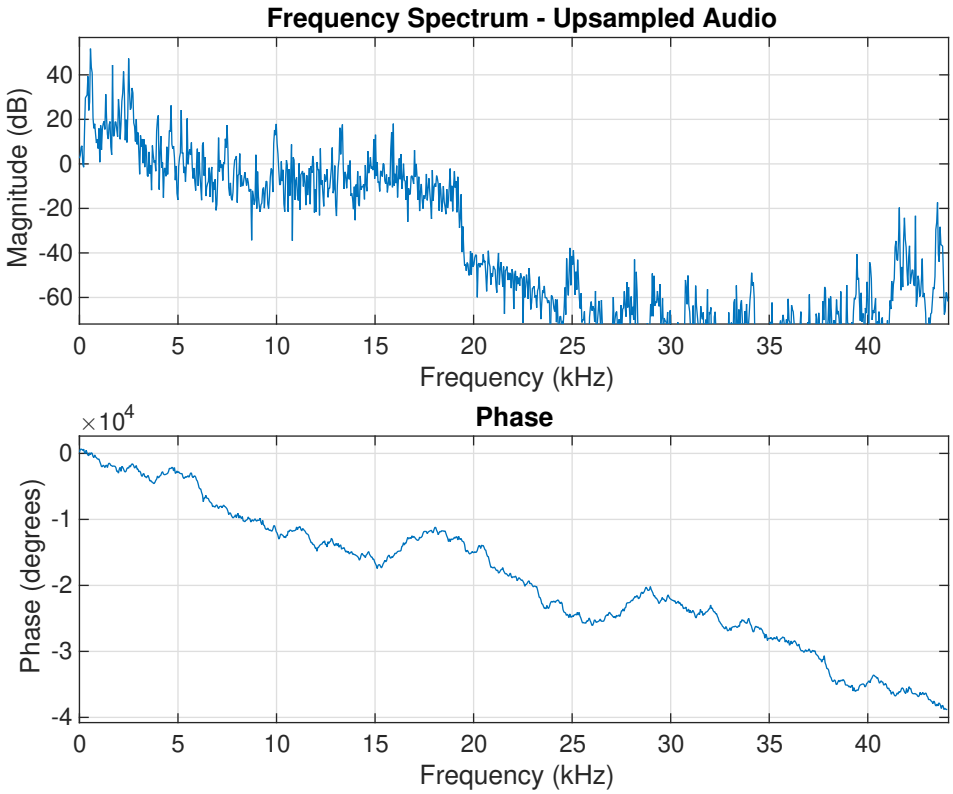
```
44100
```

Using this we were able to determine that the audio sampling frequency was 44100. We also converted the audio to mono-channel rather than stereo-channel as otherwise it would not be compatible with certain Matlab functions such as `freqz`.

2.1.2 Upsampling and viewing ths spectrum

```
% Upsameple the audio
audio_up = resample(audio, 2, 1);
Fs_new = Fs * 2;

% Plot normal and upsampled audio
figure;
freqz(audio, 1, 1024, Fs);
title('Frequency Spectrum - Original Audio');

figure;
freqz(audio_up, 1, 1024, Fs_new);
title('Frequency Spectrum - Upsampled Audio');
```




Frequency Spectrum - Upsampled Audio
Magnitude (dB)
40
20
0
-20
-40
-60
0
5
10
15
20
25
30
35
40
Frequency (kHz)
Phase
×10^4
Phase (degrees)
0
-1
-2
-3
-4
0
5
10
15
20
25
30
35
40
Frequency (kHz)

2.1.3 Adding the sinusoidal interference

In this stage we extracted the useful frequencies in the audio to be able to place the noise at a distance away from it. This was done using the Discrete Fourier Transform of the audio signal rather than extracting it visually using the plots so make the code more flexible.

```
% Getting the useful range of frequenceies to put the interfernece
    outside
% of that range
N = length(audio);
f = linspace(0, Fs/2, N/2);
Audio_FFT = abs(fft(audio));
threshold = max(Audio_FFT) * 0.05;

audio_freq_range = f(Audio_FFT(1:N/2) > threshold);

min_freq = min(audio_freq_range);
max_freq = max(audio_freq_range);
fprintf('Audio Frequency Range: %.2f Hz - %.2f Hz\n', min_freq,
    max_freq);
```

```
Audio Frequency Range: 308.13 Hz - 2679.55 Hz
```

```
% Add interfernce to normal and upsampled signals
t_up = (0:length(audio_up)-1)/Fs_new;
f_interf = max_freq*4;
interf_up = 0.5 * sin(2*pi*f_interf*t_up)';
audio_up_interf = audio_up + interf_up;
```

We have the interference placed at 4 × Maximum Useful Frequency so we can design our filter to have a cutoff of 2 × Maximum Useful Frequency.

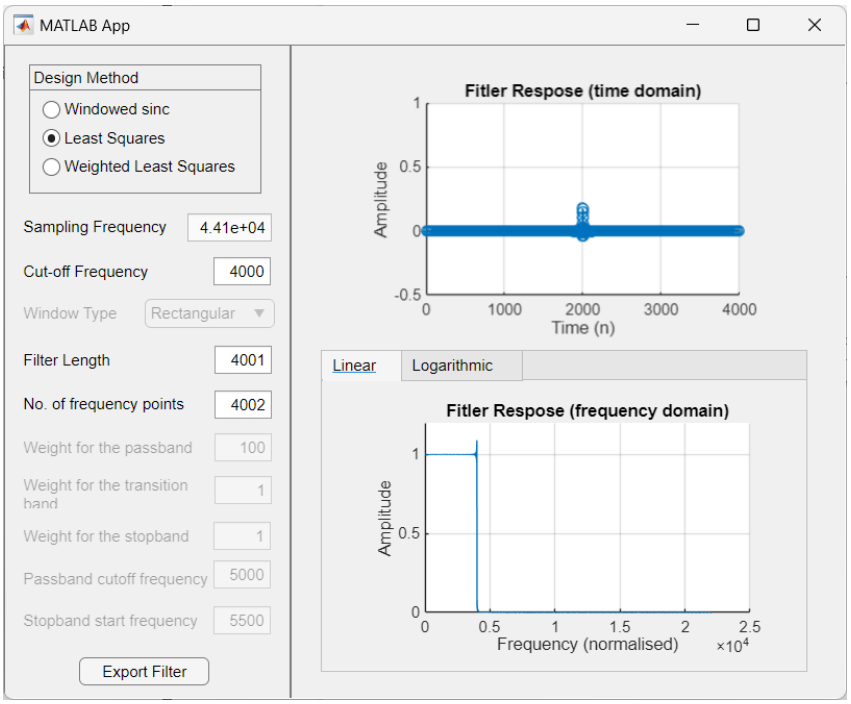


Figure 4: GUI for Weighted Least Squares

Why was it chosen?
Earlier we knew that our audio's maximum useful frequency is 2680, we placed the noise at 4 times that and hence the filter at 2 times that.

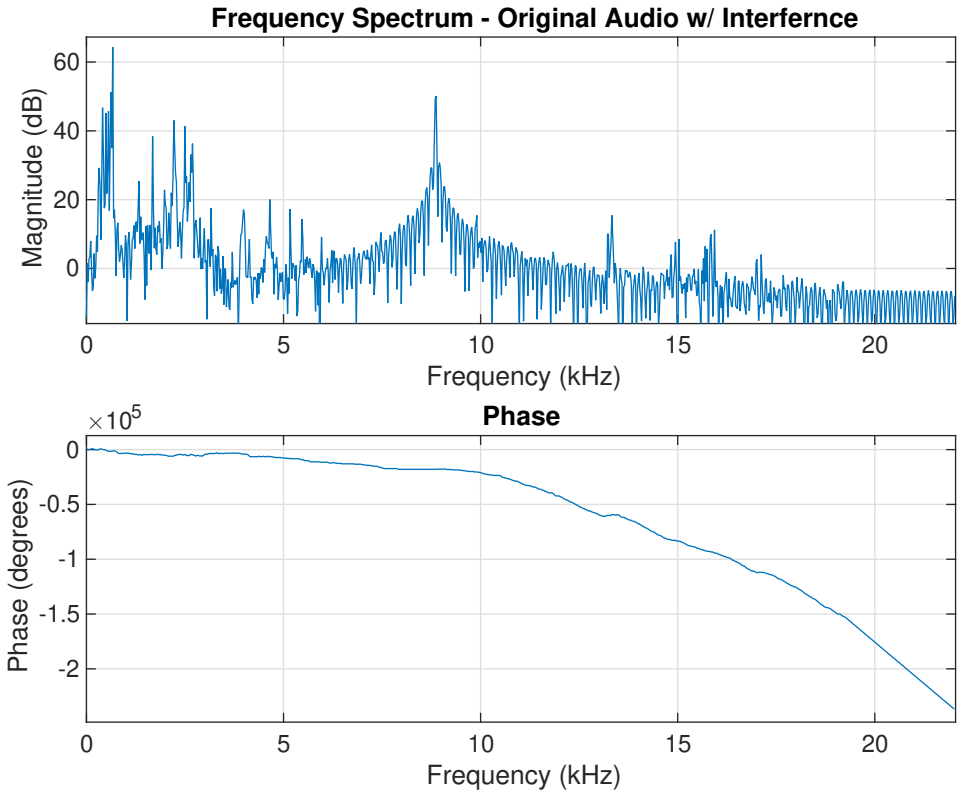
We decided to pick a middle-ground for our cutoff so we picked a frequency of 4000Hz, it is safely out of range of the useful frequencies of the audio and its far enough away from the noise to ensure the attenuation will be maximized.

We first attempted lower order (less length) windowed sinc filters however the noise was not properly removed and we needed to run it though the filter multiple times to see a good effect on the noise removal.

We then readjusted our parameters and chose weighted least squared and reached an almost ideal noise removal, where we cannot see or hear the noise anymore in our audio.

We then plot the spectrum after the noise to be able to visualize the new signal

```
% Plot normal and upsampled audio
figure;
freqz(audio_interf, 1, 1024, Fs);
title('Frequency Spectrum - Original Audio w/ Interfernce');
```



2.2 Filtering The Audio

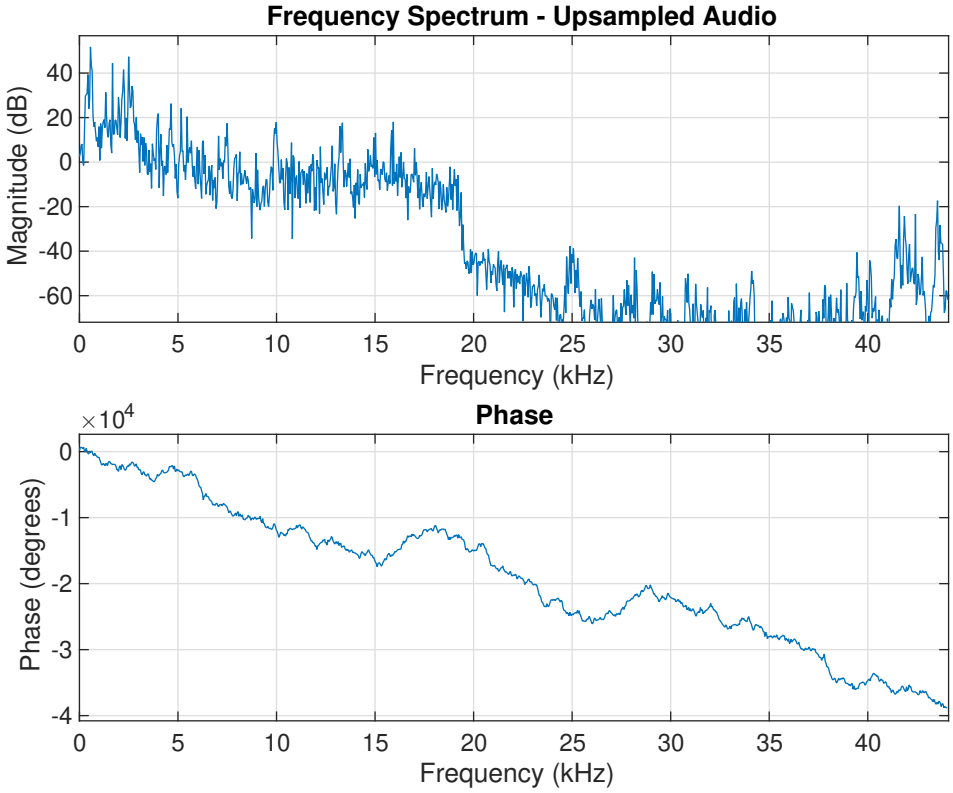
2.2.1 Importing the filter and viewing the specs

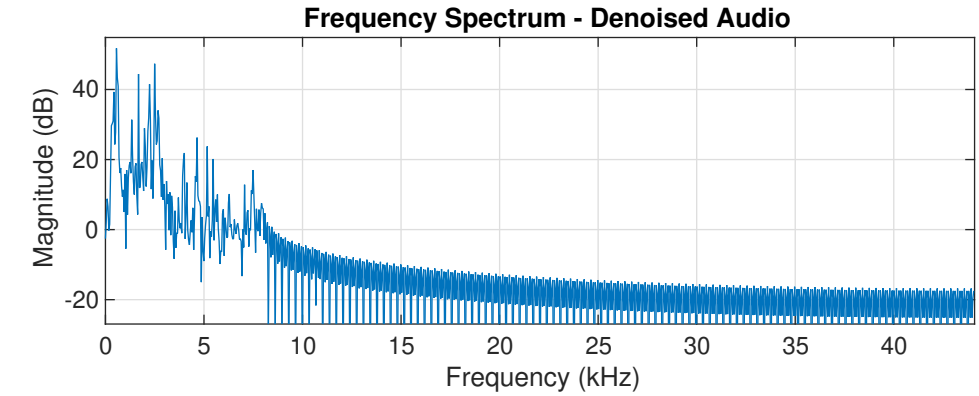
```
filter_struct = load('filter.mat').filter_specs;
filter = filter_struct.filter_time_domain;

filter_table = table(...
    filter_struct.filter_L, ...
    string(filter_struct.filter_win_type), ...
    filter_struct.filter_fs, ...
    filter_struct.filter_fc, ...
    filter_struct.filter_k, ...
    'VariableNames', {'FilterLength', 'WindowType', 'SamplingFreq', '
       CutoffFreq', 'Gain'});
disp('Filter specs: ')
disp(filter_table);
```

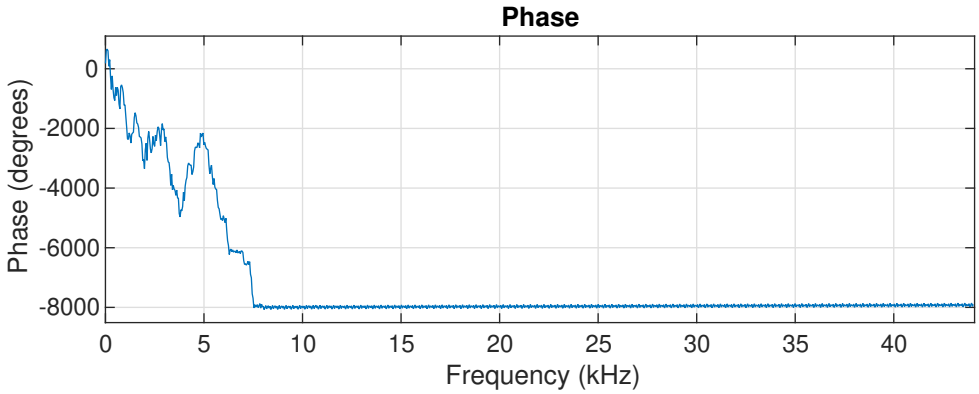
```
Filter specs:

    FilterLength     WindowType      SamplingFreq    CutoffFreq    Gain
    ------------    -------------    ------------    ----------    ----

        4001        "Rectangular"        44100           4000       4002
```

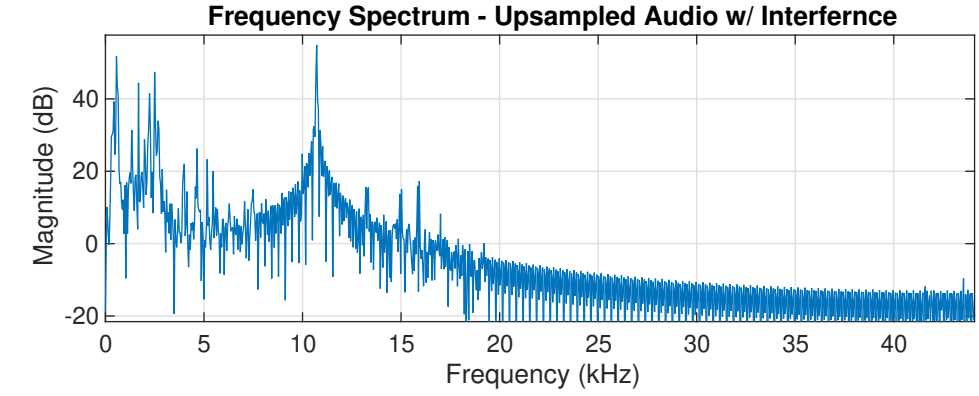
2.2.2 Applying the filter and plotting the output

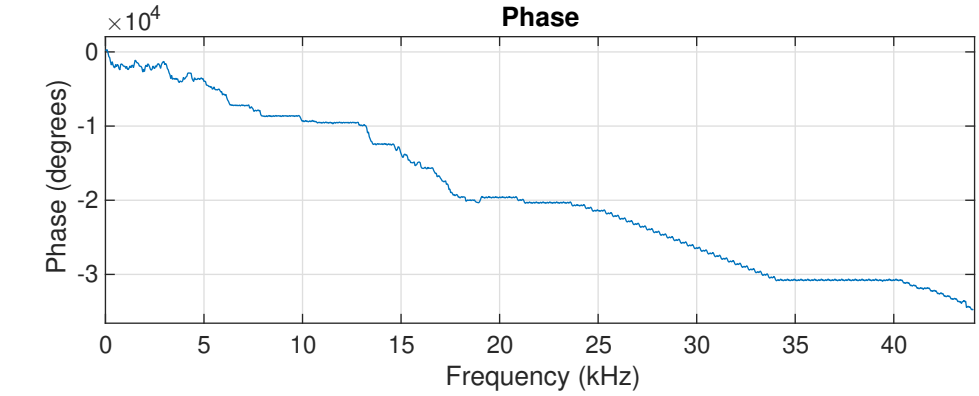
```
audio_denoised = conv(audio_up_interf, filter, 'same');

figure;
freqz(audio_up, 1, 1024, Fs_new);
title('Frequency Spectrum - Upsampled Audio');

figure;
freqz(audio_denoised, 1, 1024, Fs_new);
title('Frequency Spectrum - Denoised Audio');

figure;
freqz(audio_up_interf, 1, 1024, Fs_new);
title('Frequency Spectrum - Upsampled Audio w/ Interfernce');
```











We can see that based on the choice of our filter the noise was successfully removed and this can be further proven by listening to the audio before and after applying the filter.

3 Appendix

3.1 Code for playing the audio

```
% You can choose your audio by setting the play_sound_denoise variable
    and to start or stop the audio you can select the required section
    and run that specific selection, this code was initally written for
    Matlab Live Script hence the extra steps to isolate sections.
%%
s_u = audioplayer(audio_up, Fs_new);
s_u_n = audioplayer(audio_up_interf, Fs_new);
s_u_d = audioplayer(audio_denoised, Fs_new);

if (exist('s_u_d', 'var'))
    s_u_d.pause;
end
if (exist('s_u', 'var'))
    s_u.pause;
end
if (exist('s_u_n', 'var'))
    s_u_n.pause;
end

play_sound_denoise = 'sound_denoised'
switch (play_sound_denoise)
    case 'sound_denoised'
        s_u_d.play;
    case 'sound_w_noise'
        s_u_n.play;
    case 'sound_upsampled'
        s_u.play;
end
%%z
if (exist('s_u_d', 'var'))
    s_u_d.pause;
end
if (exist('s_u', 'var'))
    s_u.pause;
end
if (exist('s_u_n', 'var'))
    s_u_n.pause;
end
```